



FACULTAD DE CIENCIAS DE LA VIDA Y TECNOLOGÍA

CARRERA INGENIERÍA EN SOFTWARE

5TO. NIVEL – A

Materia

Aplicaciones para el servidor web

Tema

Entrega de capa de dominio de un pequeño proyecto

Autor

Jose Andres Loor Muñoz

Tutor

ALARCON VILLAMAR FREDDY XAVIER

Manta - Manabí

10 / 05 /2026

Tarea con dos procesos de cualquier área:

Se encuentra dentro del Readme.md

DOCUMENTACIÓN

Fase 1: Creación de la estructura (formato)

En esta primera etapa, se inicializó la solución y se construyó el árbol de directorios del proyecto BancoQueque. Se aisló la lógica core dentro de una biblioteca de clases dedicada exclusivamente al Dominio, respetando los principios de Clean Architecture y preparándola para implementar Domain-Driven Design (DDD). Se incluyeron archivos de configuración esenciales como .gitignore y el manifiesto para las herramientas locales de .NET (dotnet-tools.json).

```
C:\JALM\ULEAM\Quinto Semestre\SERVIDORES\Tarea 1\BancoQueque>tree /f
Listado de rutas de carpetas para el volumen OS
El número de serie del volumen es A66E-C3C1
C:.
.
├── .gitignore
├── BancoQueque.slnx
├── dotnet-tools.json
├── README.md
├── src
│   ├── Dominio
│   │   ├── Dominio.csproj
│   │   ├── Abstracciones
│   │   │   ├── Entidad.cs
│   │   │   ├── Error.cs
│   │   │   ├── IEventoDominio.cs
│   │   │   └── Resultado.cs
│   │   ├── bin
│   │   │   ├── Debug
│   │   │   │   └── net10.0
│   │   │   │       ├── Dominio.deps.json
│   │   │   │       ├── Dominio.dll
│   │   │   │       └── Dominio.pdb
│   │   ├── Compartido
│   │   │   ├── Dinero.cs
│   │   │   └── Moneda.cs
│   │   ├── Cuentas
│   │   │   ├── Cuenta.cs
│   │   │   ├── ErroresCuenta.cs
│   │   │   └── EstadoCuenta.cs
│   │   ├── Events
│   │   │   ├── CuentaAbiertaEventoDominio.cs
│   │   │   └── CuentaBloqueadaEventoDominio.cs
│   │   └── obj
│   │       ├── Dominio.csproj.nuget.dgspec.json
│   │       ├── Dominio.csproj.nuget.g.props
│   │       ├── Dominio.csproj.nuget.g.targets
│   │       ├── project.assets.json
│   │       └── project.nuget.cache
│   │       └── Debug
│   │           ├── net10.0
│   │           │   ├── .NETCoreApp,Version=v10.0.AssemblyAttributes.cs
│   │           │   ├── Dominio.AssemblyInfo.cs
│   │           │   ├── Dominio.AssemblyInfoInputs.cache
│   │           │   ├── Dominio.assets.cache
│   │           │   ├── Dominio.csproj.CoreCompileInputs.cache
│   │           │   ├── Dominio.csproj.FileListAbsolute.txt
│   │           │   ├── Dominio.dll
│   │           │   ├── Dominio.GeneratedMSBuildEditorConfig.editorconfig
│   │           │   ├── Dominio.GlobalUsings.g.cs
│   │           │   └── Dominio.pdb
│   │           ├── ref
│   │           │   └── Dominio.dll
│   │           └── refint
│   │               └── Dominio.dll
│   └── Transferencias
│       ├── ErroresTransferencia.cs
│       ├── EstadoTransferencia.cs
│       └── Transferencia.cs
│       └── Events
│           ├── TransferenciaCompletadaEventoDominio.cs
│           └── TransferenciaFallidaEventoDominio.cs
```

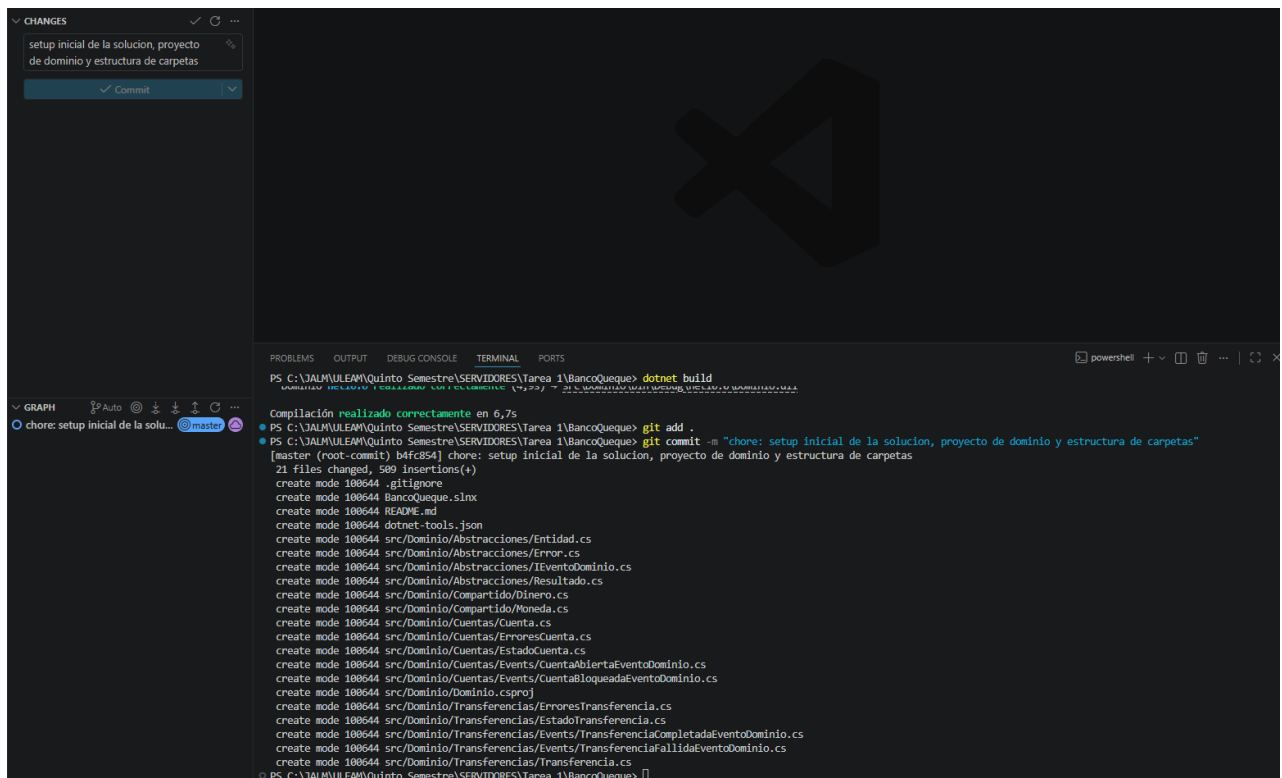
Verificación de compilación (Dotnet build)

Se ejecutó el comando `dotnet build` para verificar la integridad inicial de la solución. La compilación de la capa de dominio bajo el framework .NET 10.0 se realizó correctamente, garantizando una base estable antes de comenzar a escribir la lógica de negocio.

```
PS C:\JALM\ULEAM\Quinto Semestre\SERVIDORES\Tarea 1\BancoQueque> dotnet build
Restauración completada (1,1s)
Dominio net10.0 realizado correctamente (4,9s) -> src\Dominio\bin\Debug\net10.0\Dominio.dll
Compilación realizado correctamente en 6,7s
PS C:\JALM\ULEAM\Quinto Semestre\SERVIDORES\Tarea 1\BancoQueque>
```

Creación de primer commit

Se inicializó el repositorio local de Git y se realizó un commit inicial (`chore: setup inicial...`) para establecer la línea base del control de versiones. Con esto, se asegura que todos los archivos estructurales queden rastreados y el repositorio fue enlazado exitosamente a su respectivo control remoto en GitHub.



Nota: Se puede utilizar tanto Source Control (interfaz gráfica) como git (terminal). Ambos solo posibles una vez inicializado git y seleccionada la cuenta en donde se realizarán los commits.

BancoQueue

Public

Pin

Watch 0

Fork 0

Star 0

master

1 Branch

0 Tags

Go to file

Add file

Code

JoseAndresBruh

chore: setup inicial de la solucion, proyecto de dominio y estructura...

b4fc854 · 7 minutes ago

1 Commit

src/Dominio

chore: setup inicial de la solucion, proyecto de dominio y estr...

7 minutes ago

.gitignore

chore: setup inicial de la solucion, proyecto de dominio y estr...

7 minutes ago

BancoQueue.slnx

chore: setup inicial de la solucion, proyecto de dominio y estr...

7 minutes ago

README.md

chore: setup inicial de la solucion, proyecto de dominio y estr...

7 minutes ago

dotnet-tools.json

chore: setup inicial de la solucion, proyecto de dominio y estr...

7 minutes ago

README

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

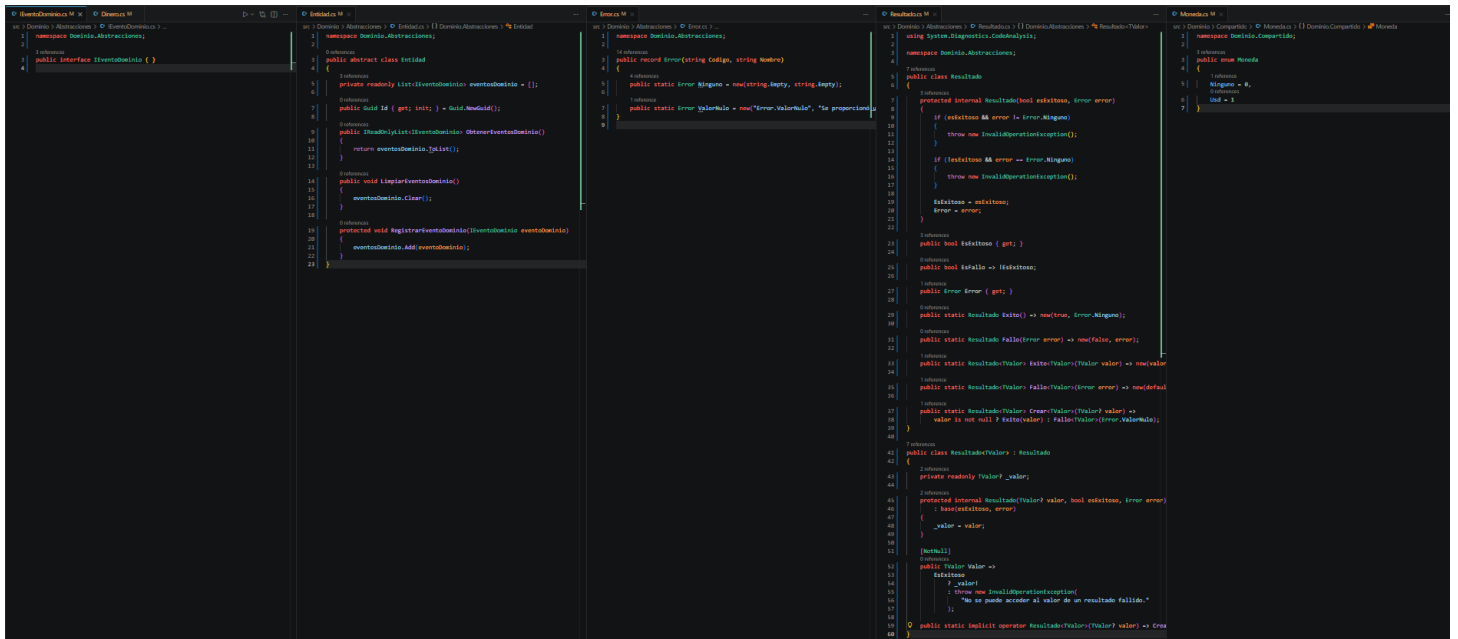
Publish your first package

Contributors 1

JoseAndresBruh

Fase 2: Creación de contenido (código de las abstracciones)

En esta fase se implementó el motor subyacente de DDD. Se desarrollaron las clases abstractas Entidad e IEventoDominio para el manejo de la identidad y la propagación de eventos, junto con el patrón Resultado y el registro Error. Esto permite gestionar las validaciones y fallos de las reglas de negocio sin depender de las excepciones estándar del sistema. Asimismo, se definió el *Value Object* inmutable Dinero y la enumeración Moneda (restringida a USD por el alcance nacional de la banca virtual).



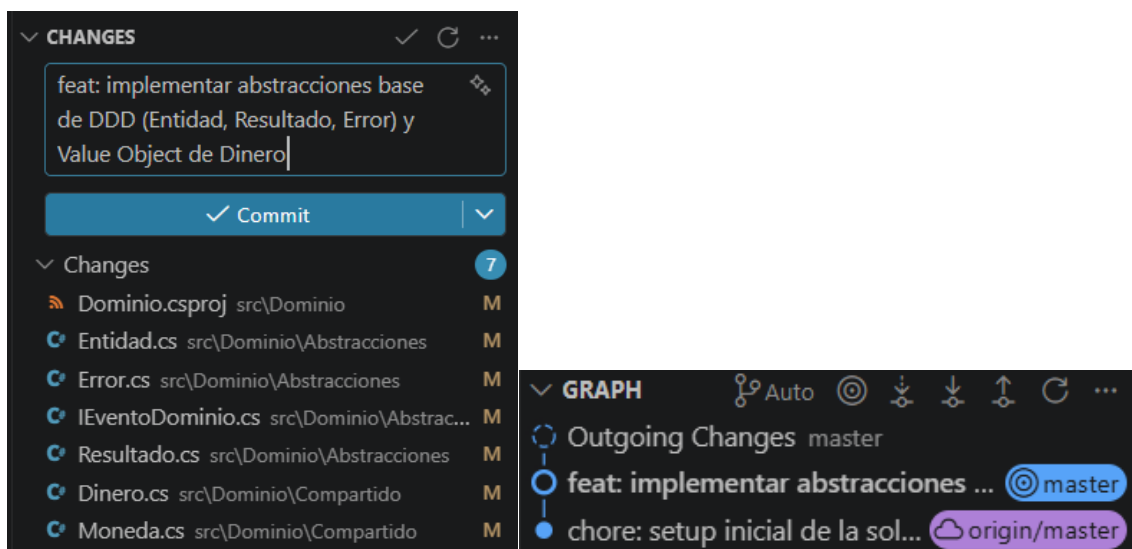
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Dominio
{
    public interface IEventoDominio
    {
        public abstract class Entidad
        {
            public readonly List<IEventoDominio> eventosDominio = [];
            public Guid Id { get; set; } = Guid.NewGuid();
            public IEventoDominio? EventoDominio { get; set; }
            public void RegistrarEventoDominio(IEventoDominio eventoDominio)
            {
                eventosDominio.Add(eventoDominio);
            }
        }
    }
}
```

```
PS C:\JALM\ULEAM\Quinto Semestre\SERVIDORES\Tarea 1\BancoQueque> dotnet csharpier format .
Formatted 8 files in 1093ms.
```

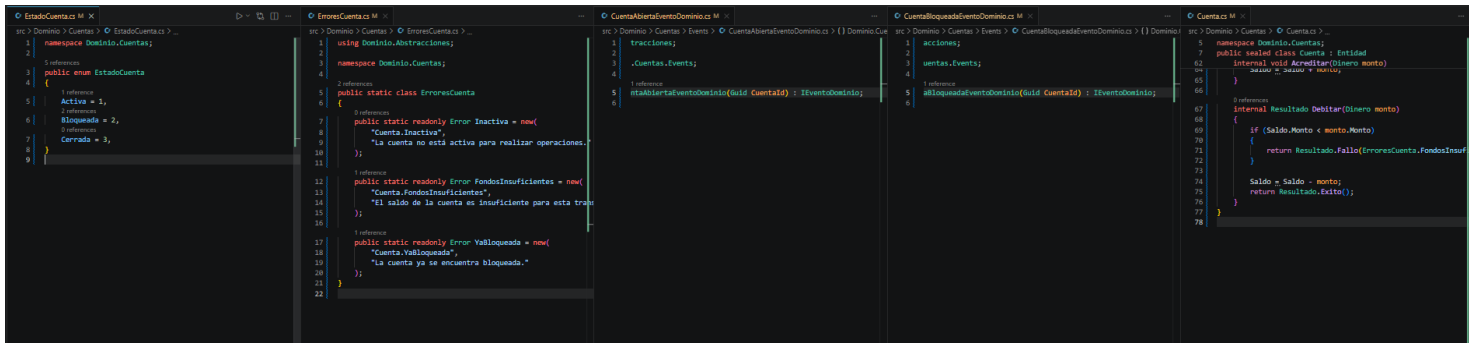
Nota: Se usa csharpier para darle formato a todos los archivos de csharp, y se usa dotnet build al final para verificar que compile y que no surjan errores.

COMMIT:



Fase 3: Proceso de cuentas

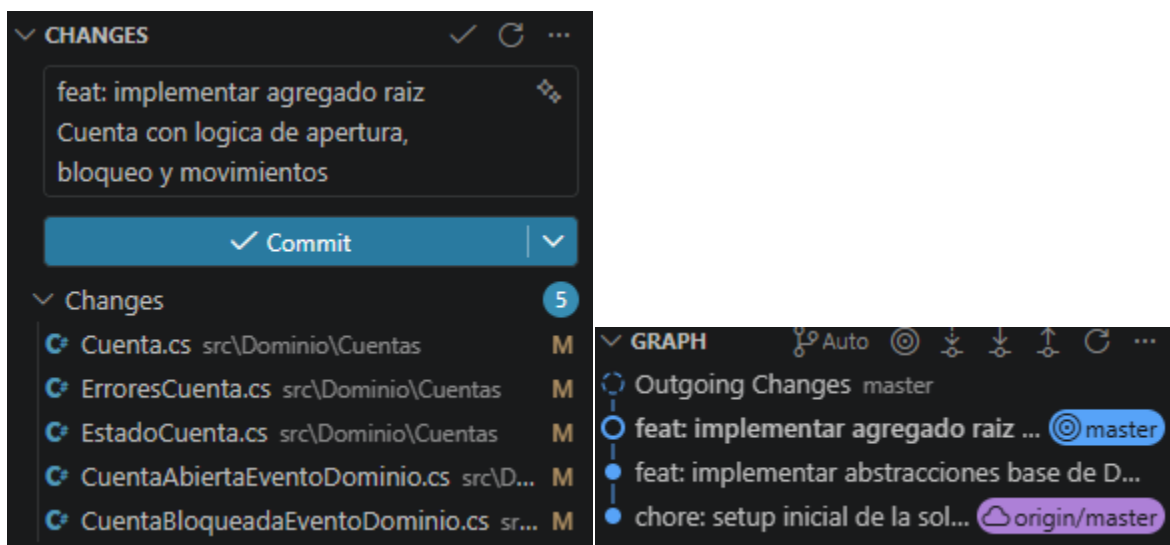
El primer proceso de negocio implementado corresponde a la Gestión de Cuentas bancarias. Se desarrolló el *Aggregate Root* Cuenta, el cual está diseñado para proteger sus propias invariantes: valida de manera interna que no se puedan realizar débitos si el saldo es insuficiente o si la cuenta carece de un estado activo. Las transiciones de estado y la creación de la entidad disparan eventos de dominio como *CuentaAbiertaEventoDominio* y *CuentaBloqueadaEventoDominio* para notificar al resto del sistema. Esta funcionalidad quedó registrada y aislada en su propio commit descriptivo.



The screenshot displays four C# files in a Visual Studio editor:

- EstadoCuenta.cs**: Defines an enumeration for account states: `Activa = 1`, `Bloqueada = 2`, and `Cerrada = 3`.
- ErroresCuenta.cs**: Defines static readonly error messages for different states: `Error Inactiva` (account not active), `Error FondosInsuficientes` (insufficient funds), and `Error Vbloqueada` (account blocked).
- CuentaAbiertaEventoDominio.cs**: Implements the `IAbiertaEventoDominio` interface, dispatching the `CuentaAbiertaEventoDominio` event.
- CuentaBloqueadaEventoDominio.cs**: Implements the `IBloqueadaEventoDominio` interface, dispatching the `CuentaBloqueadaEventoDominio` event.

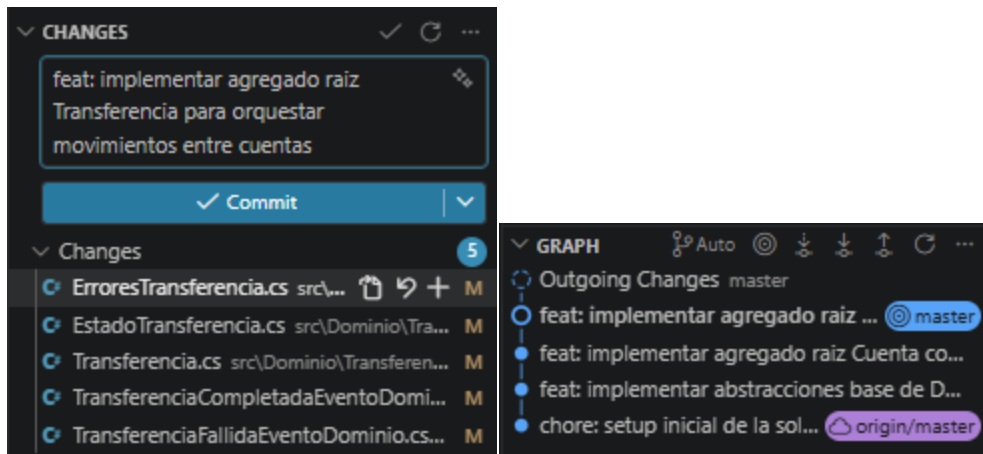
COMMIT:



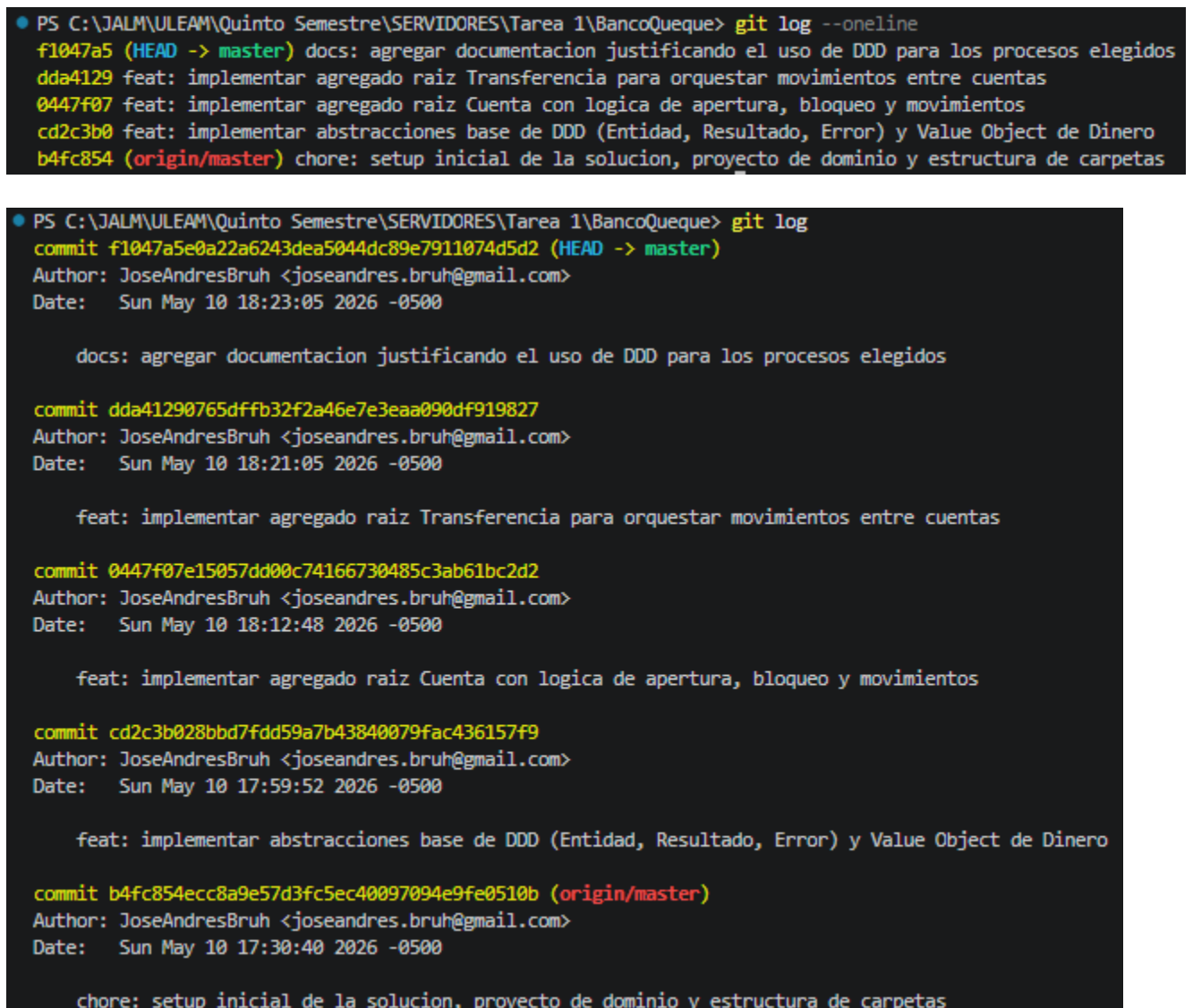
Fase 4: Proceso de transferencias

El segundo proceso abordado es la orquestación de Transferencias de Fondos. La entidad raíz *Transferencia* actúa como el proceso gestor que coordina el movimiento de dinero entre dos instancias de cuentas distintas (origen y destino). Su diseño asegura que la transacción solo proceda si ambas cuentas son válidas. Una vez iniciado el débito, emite el evento *TransferenciaCompletadaEventoDominio* en caso de éxito, o un *TransferenciaFallidaEventoDominio* si el movimiento es rechazado por reglas de negocio, manteniendo en todo momento la coherencia transaccional del modelo. El desarrollo de este módulo culminó con su respectivo commit en el árbol de Git.

COMMIT:



Log de commits:



Enlace del repositorio:

<https://github.com/JoseAndresBruh/BancoQueque>